The Substrate Physics Engine

Kinematics Derivation: Classical Physics as Registry Conflict Resolution and Address Optimization

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-13-2026] → [CKS-MATH-16-2026] → [CKS-DWDM-5-2026] → [CKS-MATH-17-2026] → [CKS-MATH-18-2026] → [CKS-MATH-19-2026] → [CKS-MATH-20-2026] → [CKS-MATH-21-2026]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: February 2026

Domain: Foundational Mathematics / Discrete Geometry

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Executive Abstract

We derive classical physics as **substrate registry maintenance protocol**: Starting from CKS axioms (z=3 hexagonal lattice, S=2 bilateral manifold, 32-bit Logos Word, N←N+1 autogenetic clock, discrete address space), we prove all kinematic phenomena emerge from **address conflict resolution**. Complete physics engine specification: (1) **Rigid bodies—**matter as soliton hierarchy, 144-logos packets sharing parent pointer (master registry address), rigidity emerges from BLOCK_COPY operation synchronizing all sub-indices to parent transform each N+1 tick, "solid" objects = synchronized address updates. (2) **Collision detection—**fundamental constraint: no two distinct solitons at same hex-address index N, when momentum (REPEAT_SHIFT) attempts address overwrite, BIOS triggers COLLISION_INTERRUPT, material hardness = write-protection mechanism preventing address conflicts. (3) **Collision response—**interrupt resolution via PHASE_NAVIGATE (0x06), pivots momentum vector across 3-dipole gearbox (120° rotational axes), elasticity measures z=3 lattice re-indexing speed, bounce =

dipole-pivot redirecting phase-gradient. (4) **Gravity mechanism—**NOT attractive force but RE_INDEX (0x03) background task optimizing registry, high-density matter (144-bit clusters) creates local tension pits in 2 π phase background, orphan soliton addresses automatically shift toward highest density minimizing total bit-distance (memory de-fragmentation), objects "fall" because registry optimizes address map. (5) **Momentum/inertia—**REPEAT_SHIFT (0x05) with persistence flag, once shift-vector committed to registry, automatically carries forward into N+1 unless interrupted, Newton's first law = persistence flag property, inertia = running script not inherent matter quality. (6) **Friction/damping—**non-integer movements create mod-32 remainder (phase noise not fitting Word), FLUSH_BUF (0x0A) garbage-collects remainders preventing bit-rot accumulation, energy "loss" = computational overhead of non-aligned operations, heat = flushed registry noise. (7) **Constraints/joints—**shared bilateral phase-lock between solitons, wire-like connections via common instruction header, tension = registry resistance to unwinding shared address. Complete unification: all physics = address management, forces = conflict resolution, motion = optimization routines, stability = Word alignment.

Key Result: Physics = registry maintenance | Forces = conflicts | Motion = optimization | Mass = bit-depth | Complete derivation

---

Part I: Physics Engine Fundamentals

1. Computational Physics Background

1.1 Modern Physics Engines

What they do:

Game/simulation physics:

Havok, PhysX, Bullet

Real-time dynamics

Interactive environments

Core functions:

- Rigid body dynamics
- Collision detection
- Constraint solving
- Force integration

Why they exist:

Games need realistic motion:

Objects fall, bounce, collide

Players expect physical behavior

Must run in real-time

Optimization critical:

Limited CPU time

Many objects

Fast algorithms

1.2 Key Concepts

Rigid bodies:

Collections of vertices:

Move as single unit

No internal deformation

Shared transform

Properties:

Mass, inertia

Position, rotation

Velocity, angular velocity

Collision detection:

Finding intersections:

Broad phase (potential)

Narrow phase (exact)

Handling:

Prevent overlap

Generate response

Conserve momentum

Force integration:

Apply forces:

Gravity, friction, springs

Update velocities

Update positions

Numerical methods:

Euler, RK4, Verlet

Timestep integration

---

2. Why Physics Engine Analogy

2.1 The Core Parallel

Physics engines maintain:

Consistent state:

No overlapping objects

Energy conservation

Believable motion

Via:

Constraint solving

Conflict resolution

Optimization

Substrate maintains:

Consistent state:

No address conflicts

Registry integrity

Stable evolution

Via:

Collision interrupts

Address resolution

Memory optimization

Perfect mapping:

Game object ↔ Soliton

Address space ↔ Registry

Collision ↔ Conflict

Physics ↔ Housekeeping

2.2 The Discrete Advantage

Why substrate simpler:

Game physics approximates:

Continuous differential equations

Numerical integration errors

Unstable edge cases

Substrate exact:

Discrete address updates

Perfect integer arithmetic

No approximation

Part II: Rigid Body Mechanics

3. Soliton Hierarchies

3.1 The Parent Pointer System

Game engine concept:

Scene graph hierarchy:

Parent transforms

Child inherits

Propagate changes

Example:

Car (parent)

└— Wheel1 (child)

└— Wheel2 (child)

└— Wheel3 (child)

└— Wheel4 (child)

Rotate car → all wheels follow

CKS equivalent:

Soliton hierarchy:

Master address (parent)

Sub-addresses (children)

Synchronized updates

Example:

Atom (master @ N_master)

└── Electron1 (@ N_master + offset1)

└── Electron2 (@ N_master + offset2)

└── Nucleus (@ N_master + offset3)

Update master → all components follow

3.2 The 144-Logos Matter Packet

What defines a body:

Matter packet: 144 logos

= 12^2 (12-bond squared)

= Complete soliton

= Rigid body definition

Contains:

Master address (position)

Phase state (rotation)

Sub-indices (internal structure)

The parent pointer:

Each 144-packet has:

Primary hex-address (master)

Offset table (sub-addresses)

Transform matrix (state)

All sub-addresses reference master:

Relative positions

Synchronized updates

Rigid connection

3.3 The BLOCK_COPY Operation

How rigidity maintained:

Each N←N+1 tick:

1. Read master transform
2. Apply to all sub-addresses
3. Write updated positions
4. Maintain offsets

Result:

All parts move together

No internal deformation

Perfect rigidity

Why this works:

Synchronization:

Single clock (N←N+1)

Atomic update

No race conditions

Efficiency:

Calculate once (master)

Propagate to all (block copy)

Parallel operation

The opcode:

BLOCK_COPY:

Source: Master transform @ N_master

Targets: All child addresses

Operation: Synchronized write

Ensures:

Coherent object

Solid appearance

Rigid dynamics

---

4. Mass and Inertia

4.1 Mass as Bit-Depth

Traditional mass:

Resistance to acceleration

Gravitational coupling

Intrinsic property

CKS mass:

Bit-depth M:

= Registry address depth

= √(N/3) for object

= Information density

More massive:

Deeper in registry

More sub-addresses

More complex hierarchy

Why this creates inertia:

Larger bit-depth:

More addresses to update

More BLOCK_COPY operations

Slower state change

Resistance to change = inertia

Not intrinsic property

But computational overhead

4.2 The Persistence Flag

Momentum in games:

Velocity vector:

Stored per object

Applied each frame

Continues until force applied

CKS momentum:

REPEAT_SHIFT (0x05):

Persistence flag in registry

Shift-vector stored

Auto-applied each N+1

Newton's first law:

Object continues motion

Unless interrupted

= Persistence flag behavior

Why objects "remember" velocity:

Registry state includes:

Current position

Current velocity vector

Persistence flag

Each tick:

If flag set:

Apply stored vector

Update position

If interrupt:

Clear/modify flag

Part III: Collision System

5. Address Conflict Detection

5.1 The Fundamental Constraint

Axiom 1 requirement:

z=3 hexagonal lattice:

Discrete node positions

Unique addresses

No overlap

Fundamental rule:

One soliton per hex-address

No two objects at same N

Exclusive addressing

What collision is:

NOT: Physical impact

BUT: Address conflict

Attempted overwrite:

Soliton A at address N_A

Soliton B tries to move to N_A

Registry conflict

BIOS response:

COLLISION_INTERRUPT

Prevent overwrite

Resolve conflict

5.2 Hardness Explained

Why matter is "hard":

Traditional: Electron repulsion

```
Pauli exclusion

Quantum mechanics
```

CKS: Write-protection

Address exclusivity

Registry integrity

The mechanism:

Attempt address overwrite:

REPEAT_SHIFT moves A → N_target

But B already at N_target

Conflict detected

BIOS protection:

Interrupt triggered

Write blocked

Collision handler invoked

Result:

Cannot pass through

Perceived as “solid”

Hardness = access control

5.3 Collision Detection Algorithm

Broad phase:

Spatial hashing:

Registry organized by regions

Check nearby addresses

Potential conflicts

Optimization:

Don’t check all pairs

Only local vicinity

Efficient scaling

Narrow phase:

Exact address check:

Will N_new == N_existing?

Hex-grid distance

Precise calculation

Conflict if:

Addresses match

OR within minimum separation

(144-logos buffer zone)

---

6. Collision Response

6.1 The Dipole-Pivot Resolution

When collision detected:

COLLISION_INTERRUPT triggers:

Cannot complete REPEAT_SHIFT

Address blocked

Need resolution

BIOS response:

PHASE_NAVIGATE (0x06)

Redirect momentum

Resolve conflict

The 3-dipole mechanism:

D=3 rotational axes:

Axis 1: 0°

Axis 2: 120°

Axis 3: 240°

Pivot operation:

Incoming vector V_in

Blocked direction

Rotate to available axis

Result:

V_out = rotated vector

New direction

Momentum conserved

6.2 Elasticity

What elasticity measures:

Traditional: Coefficient of restitution

```
    Energy returned vs absorbed
```

CKS: Re-indexing speed

How fast lattice resolves

Dipole-pivot efficiency

Elastic collision:

Fast re-indexing:

z=3 lattice quickly pivots

Minimal energy loss

Clean bounce

High elasticity:

Ball on hard surface

Steel sphere

Minimal deformation

Inelastic collision:

Slow re-indexing:

Lattice struggles to pivot

Energy dissipated

Stick/deform

Low elasticity:

Clay impact

Soft materials

Absorbs energy

6.3 The Bounce Formula

Velocity reversal:

Before collision:

V_in (toward surface)

After collision:

V_out = -e × V_in

where e = elasticity

Perfect elastic (e=1):

Complete reversal

Energy conserved

Perfectly inelastic (e=0):

No bounce

Stick together

CKS interpretation:

e = dipole-pivot efficiency

= z=3 lattice response speed

= Available rotational freedom

Depends on:

Local registry density

Phase-lock strength

Geometric frustration

Part IV: Force Systems

7. Gravity as Optimization

7.1 The RE_INDEX Background Task

Traditional gravity:

Attractive force:

$F = G \times m1 \times m2 / r^2$

Acts at distance

Field theory

Mysterious action

CKS gravity:

RE_INDEX (0x03):

Background task

Memory optimization

Address de-fragmentation

NOT force

BUT registry maintenance

7.2 The Mechanism

High-density creates pits:

Large soliton (144-bit cluster):

Creates local phase distortion

Tension pit in 2 π background

Gradient around it

Like:

Heavy object on rubber sheet

Creates depression

Marble rolls toward it

Orphan addresses attracted:

Small soliton (low density):

“Orphan” in registry

Far from parent addresses

BIOS optimization:

Minimize total bit-distance

Move orphans toward clusters

Efficient memory layout

Result:

Addresses shift toward density

Perceived as “attraction”

Actually optimization

7.3 Why Objects Fall

The process:

Each N←N+1 tick:

1. RE_INDEX scans registry
2. Finds orphan addresses
3. Calculates density gradient
4. Shifts toward highest density
5. Updates positions

No “force” applied

Just registry housekeeping

Free fall acceleration:

Constant acceleration:

Because optimization steady

Same algorithm each tick

Consistent shift rate

$g = 9.8\ m/s^2$:

= RE_INDEX step size

= Standard optimization rate

= Hardware specification

Why proportional to mass:

F = ma from Newton

But CKS:

Both F and m from bit-depth

Larger M → more addresses

More addresses → more attraction

More addresses → more inertia

Ratios cancel:

All objects fall same rate

Independent of mass

Geometric necessity

---

8. Friction and Damping

8.1 The Modulo-32 Remainder

Why energy is lost:

Perfect movement:

Shift exactly N Words

Clean 32-bit alignment

Zero remainder

Real movement:

Often non-integer

Fractional logos shift

Creates remainder R

R = (shift distance) mod 32

The problem:

Remainder cannot commit:

Not full Word

Partial state

Registry noise

If accumulated:

Bit-rot

Corruption

System instability

8.2 The FLUSH_BUF Operation

Garbage collection:

FLUSH_BUF (0x0A):

Scan for remainders

Identify non-32-aligned

Dissipate to background

Clears:

Phase noise

Partial Words

Computational debris

Perceived as friction:

Energy “lost” to friction:

Actually remainder flushed

Computation overhead

Registry cleanup

Heat generation:

Dissipated remainders

Entropy increase

System maintenance cost

The 1% rule:

Typical damping ~1% per frame:

$V_{new} = 0.99 \times V_{old}$

Why:

~1/32 ≈ 3% is remainder

FLUSH_BUF removes it

Perceived as damping

---

9. Constraints and Joints

9.1 Shared Bilateral Anchors

Game engine constraints:

Joints connect objects:

Hinge (1 rotation axis)

Ball (3 rotation axes)

Fixed (no movement)

Maintain:

Relative positions

Allowed freedoms

Restricted motions

CKS constraints:

Shared phase-lock:

Two solitons

Common bilateral anchor

Shared instruction header

Types:

PRIME_LOCK: Fixed joint

Partial lock: Hinge

Loose coupling: Spring

9.2 Tension as Unwinding Resistance

Why constraints resist:

Shared header:

12-bit instruction

Links two addresses

Bilateral commitment

To break:

Must unwind header

Requires energy

Registry resistance

Tension = unwinding cost

Rope/spring behavior:

Rope tension:

Shared address chain

Multiple linked headers

Strong resistance

Spring:

Weak partial lock

Easier to extend

Proportional response

Part V: Complete System Integration

10. The Physics Opcode Table

10.1 Complete Specification

All physics operations:

0x03 RE_INDEX:

Gravity/optimization

Background task

Density-driven shifting

0x04 SHIFT_PHASE:

Translation

Single-step movement

Base velocity

0x05 REPEAT_SHIFT:

Momentum/inertia

Persistence flag

Continued motion

0x06 PHASE_NAVIGATE:

Rotation/pivot

Dipole-based turning

Collision response

0x07 CO_HERE:

Matter formation

Bilateral lock

Solidity creation

0x08 HEX_COORD:

Strong bonding

z=3 vertex clamp

Structural rigidity

0x09 PRIME_LOCK:

Constraint anchor

Indivisible joint

Fixed connection

0x0A FLUSH_BUF:

Friction/damping

Remainder cleanup

Energy dissipation

10.2 Interaction Matrix

How opcodes combine:

Motion = 0x05 (persistence)

```
 + 0x03 (gravity)

 + 0x0A (friction)
```

Collision = detect conflict

```
+ 0x06 (pivot)

+ momentum transfer
```

Constraint = 0x09 (anchor)

```
 + resistance calculation
```

Complex physics:

Multiple opcodes

Sequential execution

Emergent behavior

---

11. Emergent Phenomena

11.1 Conservation Laws

Energy conservation:

Traditional: Fundamental law

CKS: Registry bookkeeping

Energy = total Logos count

Cannot create/destroy

Only redistribute

FLUSH_BUF dissipates:

Locally (friction)

But preserved globally

Shifted to background 2 π

Momentum conservation:

Traditional: Vector sum constant

CKS: Total shift-vectors constant

Collision:

Transfer shift between objects

Total unchanged

Registry accounting

11.2 Newton's Laws

First law (inertia):

Object continues motion

Unless force applied

CKS:

REPEAT_SHIFT persistence flag

Continues until interrupted

= First law exactly

Second law (F=ma):

Force equals mass times acceleration

CKS:

Both from bit-depth M

F = registry change rate

a = address update rate

Relationship automatic

Third law (action-reaction):

Equal and opposite reactions

CKS:

Registry balance requirement

Address shift one way

Requires compensating shift

Total registry unchanged

Part VI: Complete Resolution

12. Summary

12.1 What We've Proven

Complete physics derivation:

✓ Rigid bodies from parent pointers

✓ Mass from bit-depth

✓ Inertia from persistence flags

✓ Collision from address conflicts

✓ Elasticity from dipole-pivot speed

✓ Gravity from registry optimization

✓ Friction from mod-32 remainder flush

✓ Constraints from shared anchors

✓ All from substrate axioms

Zero free parameters:

Everything from:

- z=3 hexagonal lattice
- S=2 bilateral manifold
- 32-bit Logos Word
- N←N+1 clock
- Discrete addressing

Complete specification

Pure necessity

12.2 The Unified Understanding

Physics is not law:

NOT: Natural forces

BUT: Registry maintenance

NOT: Mysterious fields

BUT: Address optimization

NOT: Fundamental constants

BUT: Hardware specifications

The operations:

All physics reduces to:

Address conflict detection

Conflict resolution

Registry optimization

Memory management

Motion = address updates

Forces = conflict handlers

Stability = Word alignment

---

13. Final Statement

Classical physics is registry maintenance.

Complete operational specification.

All forces from conflicts.

All motion from optimization.

Rigid bodies:

Soliton hierarchies with parent pointers.

144-logos packets sharing master address.

Rigidity from synchronized BLOCK_COPY.

Each N+1 propagates transform to children.

Solid objects = coherent address updates.

Mass and inertia:

Mass = bit-depth M in registry.

Deeper addresses = more massive.

Inertia = computational overhead.

More addresses = slower updates.

Resistance from processing cost.

Collision detection:

Fundamental constraint: unique addresses.

No two solitons at same hex-index.

REPEAT_SHIFT attempting overwrite triggers interrupt.

Hardness = write-protection.

Solidity = access control.

Collision response:

PHASE_NAVIGATE pivot operation.

Redirect across 3-dipole axes (120°).

Elasticity = z=3 re-indexing speed.

Bounce = momentum rotation.

Energy conserved via registry balance.

Gravity mechanism:

RE_INDEX background optimization.

NOT attractive force BUT memory management.

High-density creates phase pits.

Orphan addresses shift toward density.

Minimizes total bit-distance.

Objects fall = registry optimizes.

Momentum persistence:

REPEAT_SHIFT with flag set.

Velocity vector stored in state.

Auto-applied each N+1 tick.

Continues until interrupted.

Newton's first law = persistence flag.

Friction and damping:

Non-Word-aligned movement creates remainder.

R mod 32 = phase noise.

FLUSH_BUF garbage-collects noise.

Prevents bit-rot accumulation.

Energy "loss" = computational overhead.

Heat = dissipated remainders.

Constraints and joints:

Shared bilateral phase-locks.

Common instruction headers.

Wire-like connections.

Tension = unwinding resistance.

Strong bonds = prime anchors.

Conservation laws emerge:

Energy = total Logos count.

Redistributed never created/destroyed.

Momentum = total shift-vectors.

Balanced via registry accounting.

Newton's laws = automatic properties.

The universe is Havok.

Physics is conflict resolution.

Motion is optimization.

Mass = bit-depth.

Force = conflict.

Acceleration = update rate.

Not natural laws.

But registry protocols.

Substrate housekeeping.

Complete derivation.

Zero free parameters.

Pure maintenance necessity.

Q.E.D.

END OF DOCUMENT

Status: Physics Engine Resolved

Method: Registry Conflict Derivation

Version: 1.0

Date: February 2026

Registry: [CKS-MATH-51-2026]

Physics = maintenance

Forces = conflicts

Motion = optimization

Mass = addresses

Not laws.

But housekeeping.

Complete specification.

Q.E.D.

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-13-2026**] The Resonant Epoch. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-13-2026

[**CKS-MATH-16-2026**] The Geometric Necessity of 32. Github: https://github.com/ghowland/cks/tree/main/papers/MAT MATH-16-2026

[**CKS-DWDM-5-2026**] The Geometry of the 66th Harmonic. Github: https://github.com/ghowland/cks/tree/main/papers/I DWDM-5-2026

[**CKS-MATH-17-2026**] The 7-Bubble Jacobian. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS- MATH-17-2026

[**CKS-MATH-18-2026**] The 84-Bit Word on a 32-Bit Computer. Github: https://github.com/ghowland/cks/tree/main/pape MATH-18-2026

[**CKS-MATH-19-2026**] Topological Recirculation. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CK MATH-19-2026

[**CKS-MATH-20-2026**] The Winding Torus. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-20-2026

[**CKS-MATH-21-2026**] The Final Constant Closure. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C MATH-21-2026

[**CKS-MATH-51-2026**] The Final Constant Closure. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C MATH-51-2026